

基于 xv6-RISC-V 的 静态优先级调度与 Aging 防饥饿机制

操作系统设计综合实践课程汇报

天津大学软件学院 · 第 35 组

先明确问题，再分工实现

问题与目标

xv6 原有轮转调度把可运行进程近似等价对待，难以表达任务之间的紧急程度。

- 用静态优先级表达调度倾向；
- 保留同优先级进程的基本公平；
- 用 Aging 降低低优先级进程长期等待的风险。

统一约定

0 最高

10 默认

31 最低

五人分工

赖弈铭

Aging、等待时间累计与防饥饿逻辑

刘文睿

静态优先级调度与同级公平

吴雨桐

测试设计、集成与回归验证

党正清

优先级系统调用与用户态 `nice`

袁晟棋

进程字段、默认值、fork 继承与集成

实现边界：保持 xv6 原有状态转换与锁语义，不引入 MLFQ、CFS 或复杂队列。

从用户命令

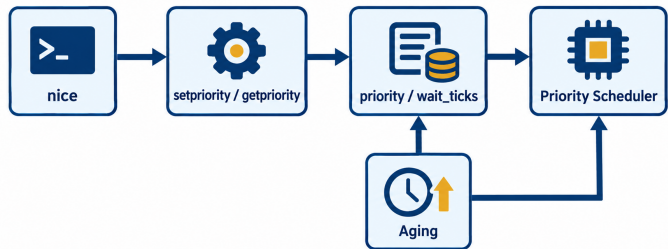


图 1 优先级控制路径：用户态命令进入系统调用，最终影响进程字段与调度选择。

优先级语义

数字越小，优先级越高；合法范围为 0-31，默认值为 10。

关键状态

`priority` 描述当前静态优先级，`wait_ticks` 为 Aging 提供等待依据。

调度入口

`scheduler` 只选择 `RUNNABLE` 进程，并在真正切换前再次确认状态。

进程优先级数据结构

字段设计

<code>priority</code>	静态优先级，合法范围 0–31。
<code>wait_ticks</code>	进程处于 <code>RUNNABLE</code> 状态时的等待累计。

统一语义

0 最高

10 默认

31 最低

生命周期处理

创建进程

设置默认优先级，清零 `wait_ticks`。`fork` 子进程

继承父进程优先级，等待计数重新置零。

调度成功

被选中的进程清零等待计数。

释放并复用

重置字段，避免旧状态影响新进程。

锁保护

`fork` 读取父进程优先级前先取得父进程锁，复制受保护的优先级快照。

静态优先级调度

两轮扫描

第 1 轮	扫描进程表；对等待进程执行 Aging，并找出最小合法优先级。	先确定需要服务的优先级层。
第 2 轮	从 <code>next_index</code> 开始循环，选择该优先级下的第一个 <code>RUNNABLE</code> 进程。	再决定这一轮由谁运行。
切换前	重新检查状态并获得进程锁，切换后推进 <code>next_index</code> 。	适配多 CPU，减少状态竞争和同级偏爱。

核心判断

$$p_{\text{best}} = \min\{p.\text{priority} \mid p.\text{state} = \text{RUNNABLE}\}$$

数值越小，进程越早获得调度机会。

公平性与 Aging

`next_index` 只在同优先级候选者之间轮转；等待达到 `AGING_INTERVAL=50` 后，优先级向 0 方向提升一级。

系统调用与用户态命令

接口链路

<code>setpriority</code>	设置目标 PID 的优先级。
<code>getpriority</code>	查询目标 PID 的优先级。
系统调用号	23 与 24, 沿用 xv6 的注册流程。
用户态入口	在 <code>user.h</code> 、 <code>usys.pl</code> 中保持一致。

内核实现

`sysproc.c` 负责取参数, `proc.c` 在遍历进程表时逐项加锁, 成功后释放锁再返回。

参数检查

- PID 必须是正数, 且目标进程存在;
- 优先级必须满足 $0 \leq p \leq 31$;
- 手动设置后清零 `wait_ticks`, 避免旧等待历史立即触发 Aging。

命令形式

`nice pid` 查询优先级; `nice pid p` 设置优先级并打印结果。

调度统计

测试输出的指标

METRIC	prioritytest 输出 case、PID、优先级、起止 tick 与 work。	比较同级公平和不同优先级的完成先后。
AGING_RESULT	输出提升耗时、目标优先级、存活竞争者和超时标志。	直接判断 Aging 是否生效。
wait_ticks	仍是内核中的等待累计字段，不新增持久化调度次数。	保持实现边界清晰，统计由测试侧汇总。

当前实现边界

内核保留 `priority` 与 `wait_ticks` 两个核心字段；调度次数和运行时间不写入 `struct proc`，避免扩大课题范围。

实际回归

单 CPU 输出 6 条 METRIC，3 CPU 输出 12 条；两组 `agingtest` 和 `usertests -q` 均通过。

项目仓库：github.com/Aphrodite-deepblue/os-design-practice

用户态测试

测试矩阵

<code>prioritytest</code>	API 边界、nice、fork 继承、同级与混合优先级。
<code>agingtest</code>	8 个高优先级竞争者存在时，低优先级进程逐级提升。
<code>usertests -q</code>	进程退出后继续运行，检查原有 xv6 行为稳定性。

程序与日志

`tests/run_scheduler_tests.py` 构建镜像、启动 QEMU，并保存原始日志、`results.json` 与 `metrics.csv`。

当前状态

单 CPU 与 3 CPU 均为 PASS；Aging 分别在 136 与 45 ticks 内完成，且未发生超时。

结果来自当前 main 快照的真实 QEMU 回归；不将一次运行当作性能基准。

谢 谢 大 家

第 35 组 · 天津大学软件学院